DATA WAREHOUSING

mining, aggregation, presentation

THE CONCEPT

Why a Data Warehouse?

Operational systems are built to run the business, not to explain it. A data warehouse pulls scattered records into one place, reshapes them into something query-able, and hands analysts a single source of truth.

- **Consolidation**

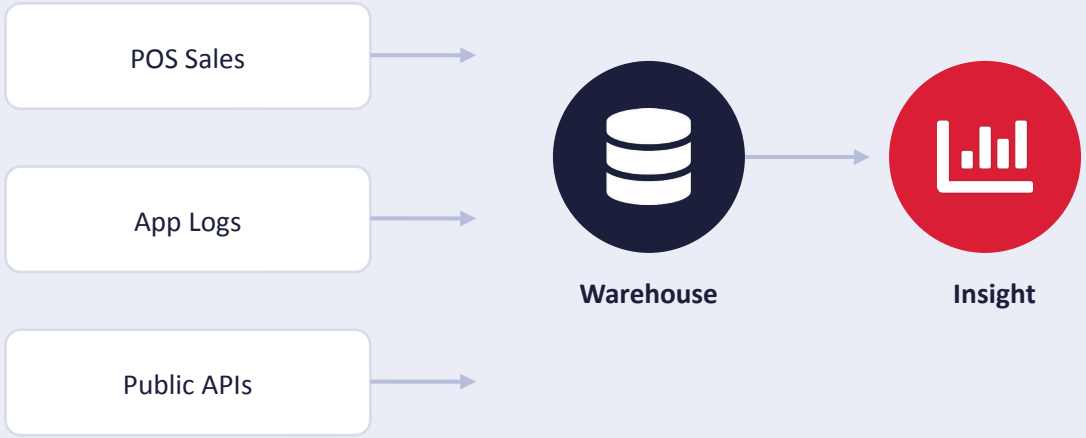
  Many source systems, one queryable model

- **Consistency**

  Cleaned, deduplicated, and standardized

- **Speed**

  Pre-shaped for fast, repeatable analysis



The 3-stage pattern shows up in almost every warehouse, from a spreadsheet-based one to a petabyte cloud platform: mine it, aggregate it, present it.

THE PIPELINE

Three Stages, One Pipeline

01

Data Mining

Locate, extract, and pull raw records out of source systems, files, and APIs.

02

Aggregation

Clean, join, group, and summarize records into analysis-ready tables.

03

Presentation

Surface the results through dashboards, reports, and interactive apps.

STAGE 1 OF 3

Data Mining

The pipeline starts by reaching into wherever data already lives, on a database, an app log, a spreadsheet, or a public API, and pulling it out in its raw, native shape.

Source systems

Transactional DBs, event streams, flat files, third-party APIs

Extraction methods

Batch pulls, incremental loads, scraping, log shipping

What changes

Nothing yet — raw grain and raw values are preserved as-is

Typical tools

DuckDB read_csv/read_json, Airbyte, Fivetran, custom scripts

Goal: get the data out, not clean it up — that comes next.

STAGE 2 OF 3

Aggregation

Raw records are cleaned, joined across sources, and rolled up — grouped, summed, averaged — into compact tables shaped around the questions people actually ask.

Cleaning

Deduplicate, standardize types, handle missing values

Joining

Combine tables across sources on shared keys

Summarizing

GROUP BY, window functions, rollups by time or category

Typical tools

DuckDB SQL, dbt, Pandas, Spark, warehouse stored procs

Goal: turn row-level detail into analysis-ready summary tables.

STAGE 3 OF 3

Presentation

Aggregated tables are the input to whatever the audience actually looks at: a chart, a dashboard, a scheduled report, or an interactive app they can filter themselves.

Static reporting

Scheduled exports, PDFs, slide decks

Dashboards

Live charts refreshed on a schedule or on demand

Interactive apps

Filters and drill-downs users control themselves

Typical tools

Streamlit, Tableau, Power BI, Looker, Metabase

Goal: make the summary legible and actionable at a glance.

CASE STUDY

The DuckDB + Streamlit Stack

A single embedded SQL engine plus a lightweight Python app framework can carry all three stages end to end — no server or cluster required.

Why it works: DuckDB runs in-process (no server to manage) and queries files directly; Streamlit turns a plain Python script into a shareable web app. Together they cover mining through presentation in well under 100 lines of code.

CASE STUDY · MINING & AGGREGATION

Indonesia's Q2 2026 Regional Growth

BPS reported 5.29% y/y GDP growth for Q2 2026. DuckDB mines the per-region release, then aggregates it by island group.

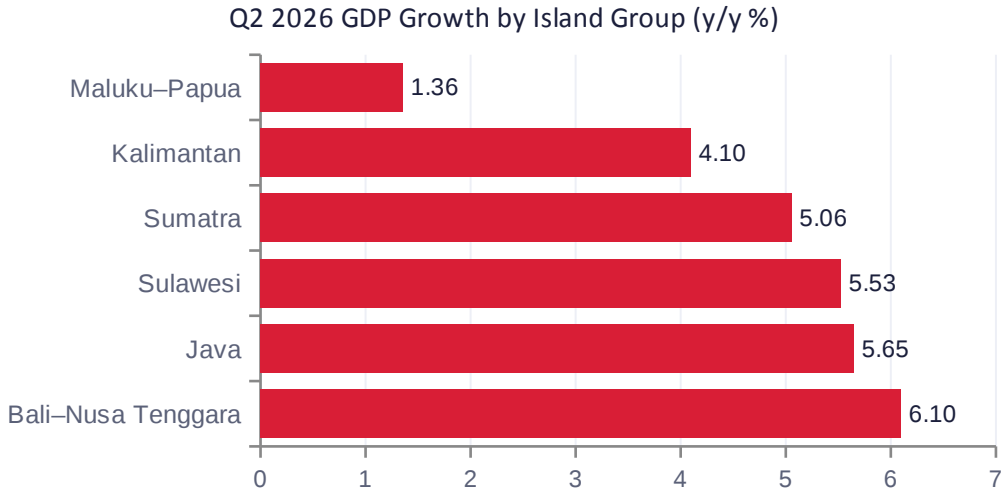
agg_gdp_by_island.py

```python
import duckdb

con = duckdb.connect("id_dw.duckdb")

# -- MINE: pull the raw BPS release --
con.sql("""
  CREATE TABLE raw_gdp AS
  SELECT * FROM read_csv_auto(
    'bps_gdp_q2_2026.csv')
""")

# -- AGGREGATE: roll up by island --
con.sql("""
  CREATE TABLE agg_by_island AS
  SELECT island,
    ROUND(AVG(yoy_growth),2) AS growth
  FROM raw_gdp GROUP BY island
  ORDER BY growth DESC
""")
```



National average: 5.29%. Maluku–Papua lagged at 1.36%, while Bali–Nusa Tenggara led at 6.10% — exactly the kind of gap a rolled-up table surfaces instantly.

Source: Statistics Indonesia (BPS), via Antara News, Aug 2026

Shipping It as a Streamlit App

Nine lines of Python turn the aggregated DuckDB table into a shareable, interactive dashboard.

dashboard.py

```python
import streamlit as st
import duckdb

con = duckdb.connect("id_dw.duckdb")
df = con.sql(
  "SELECT * FROM agg_by_island"
).df()

st.title("Indonesia Regional Growth")
region = st.selectbox(
  "Island group", df.island)
st.bar_chart(
  df.set_index("island")["growth"])
```

Interactive: filter by island group, hover for exact values, download as CSV.

Same query, same numbers — Slide 8's chart and this app read from the same aggregated table.

KEY TAKEAWAYS

One Pipeline, Three Jobs

Mine it

Pull raw records out of wherever they live — files, APIs, logs.

Aggregate it

Clean, join, and roll it up into tables shaped for real questions.

Present it

Put the summary in front of people through a chart, report, or app.

The DuckDB (backend) + Streamlit (frontend) stack